City-Wide ANPR Trajectory Tracking Platform

High-Performance AI Multi-Camera Surveillance, Spatio-Temporal Route Reconstruction & Traffic Analytics

System Version: v1.0.0 Production	**Database:** Spatio-Temporal SQLite / Async SQLAlchemy
Backend Engine: FastAPI / WebSockets (ASGI)	**Vision Stack:** YOLOv8 Detection + Multimodal Vision AI OCR

1. Executive Summary & Core Capabilities

The **City-Wide ANPR Trajectory Tracking Platform** is an enterprise distributed surveillance and traffic intelligence system designed for municipal authorities, smart city command centers, and law enforcement agencies. It delivers real-time vehicle plate recognition across dense camera topologies, chronological spatio-temporal route reconstruction, velocity violation auditing, automated clone-plate detection, and city-wide congestion heatmaps.

2. System Architecture & High-Level Data Flow

- **Edge AI Vision Ingestion:** ANPR surveillance nodes capture feeds, detect license plates via YOLOv8, and run Multimodal Vision OCR to extract alphanumeric strings with high confidence.
- **High-Concurrency Ingestion API:** Detections stream into POST /api/detections/ingest, validating schema, logging timestamps, and writing to indexed SQLite time-series tables.
- **Watchlist & OCR Fuzzy Matching:** Detected plates are instantly matched against active hotlists using exact string matching and visual OCR confusion tolerance (e.g. 0/O, 1/I, 8/B).
- **Spatio-Temporal Trajectory Engine:** Reconstructs chronological vehicle hops between cameras, computing Haversine geodesic distances, transit times, average velocities, and clone-plate anomalies.
- **Traffic Analytics & Congestion GIS:** Computes junction flow rates (vehicles/hour), 24-hour traffic density trends, and GeoJSON heatmaps for map visualizations.
- **Dual Real-Time WebSockets:** Pushes live detection feeds over /ws/live and instantaneous priority law enforcement notifications over /ws/alerts.

3. Spatio-Temporal Database Schema

Table Name	Primary Fields & Types	Indices & Constraints	Purpose & Role
cameras	id (PK), name, latitude, longitude, zone, direction, status	PK: id	Surveillance node coordinate registry and corridor topology
detections	id (PK), plate_number, camera_id (FK), timestamp, confidence, crop_path, vehicle_type	ix_plate_time (plate, ts) ix_cam_time (cam, ts)	Time-series ANPR detection logs indexed for sub-millisecond retrieval
watchlist	plate_number (PK), reason, severity, notes, created_at	PK: plate_number	Active hotlist of wanted, stolen, or flagged suspect vehicles
alerts	id (PK), plate_number, camera_id (FK), timestamp, reason, severity, match_type, acknowledged	ix_alerts_plate ix_alerts_timestamp	Audit trail of security alerts and law enforcement triggers

4. Core Algorithmic Engines

A. Trajectory Reconstruction & Haversine Velocity Analysis

For any vehicle plate, consecutive camera sightings at (lat1, lon1, t1) and (lat2, lon2, t2) are evaluated chronologically. Geodesic distance d is calculated via the Great-Circle Haversine formula (Earth radius R = 6371.0 km):

```
d = 2 * R * arcsin( sqrt( sin^2(delta_lat / 2) + cos(lat1) * cos(lat2) * sin^2(delta_lon / 2) ) )
Average Speed (km/h) = distance_km / ( (t2 - t1)_seconds / 3600 )
```

Speed Anomaly & Clone Flagging: If calculated velocity exceeds 160 km/h in an urban corridor or if two detections occur almost simultaneously at distant camera nodes (delta_t < 2s, distance > 1km), the engine automatically flags the hop with anomaly_speed = True (indicative of cloned license plates).

B. Watchlist Fuzzy Matching with OCR Confusion Matrix

Plate OCR readings frequently encounter visual ambiguities under varying lighting or angles. The alert manager implements a specialized confusion mapping (0 <-> O, 1 <-> I, 8 <-> B, 5 <-> S, 2 <-> Z) combined with Levenshtein edit distance (threshold <= 1) to achieve high true-positive detection of suspect vehicles.

5. Complete REST & WebSocket API Specification

Method	Endpoint Route	Description & Purpose	Key Output / Payload
POST	/api/detections/ingest	Camera edge ingestion; runs watchlist check & live broadcast	Detection record + Alert status
GET	/api/trajectory/{plate}	Full chronological multi-hop trajectory reconstruction	Hops, distance, speed, route GeoJSON
GET	/api/analytics/summary	Real-time city surveillance KPI dashboard summary	Detections, unique plates, active cams
GET	/api/analytics/congestion	Per-camera flow rate, top 5 junctions, 24h trends	VPH rates, hourly curve, vehicle classes
GET	/api/analytics/heatmap	GIS GeoJSON FeatureCollection with density weights	Point features with intensity weights
POST/GET	/api/watchlist	Create, update, and list active target hotlist vehicles	List of flagged plates and severity
GET	/api/cameras	List all registered ANPR surveillance cameras	Array of camera coordinate nodes
WS	/ws/live	Real-time WebSocket stream of all city-wide detections	Live detection event stream
WS	/ws/alerts	High-priority WebSocket stream for instant security alerts	Instant watchlist alert broadcasts

6. Quickstart & Verification Guide

1. **Dependencies Installation:** pip install fastapi uvicorn websockets sqlalchemy aiosqlite httpx pytest reportlab
2. **Run Full Test Suite:** python -m pytest tests/ -v (15/15 Passed)
3. **Start Development Server:** uvicorn backend.app:app --host 0.0.0.0 --port 8000 --reload
4. **Interactive Web Endpoints:**
   - Web Dashboard: http://localhost:8000/
   - OpenAPI Docs (Swagger UI): http://localhost:8000/docs
   - Live Detections WebSocket: ws://localhost:8000/ws/live
   - Security Alerts WebSocket: ws://localhost:8000/ws/alerts